Binance Futures Order Bot

Technical Execution, Architectural Analysis & Verification Report

1. Executive Summary

This report documents the architectural design, algorithmic strategy modeling, security controls, and verification results for the **Binance USDT-M Futures Order Bot**. The application is built using Python 3.10+ and strictly adheres to Binance Futures REST API standards.

2. Scope & Implementation Matrix

Component	Implementation File	Status & Highlights
Market Orders	`src/market_orders.py`	Instant execution with input validation.
Limit Orders	`src/limit_orders.py`	Limit price quantization to tickSize.
Stop-Limit Orders	`src/advanced/stop_limit.py`	Conditional order triggering at stopPrice.
OCO Engine	`src/advanced/oco.py`	Linked TP/SL orders with polling cancellation.
TWAP Strategy	`src/advanced/twap.py`	Time-weighted order chunking & scheduling.
Grid Strategy	src/advanced/grid_strategy.py	Buy-low / sell-high multi-level limit grid.
Fear & Greed	`src/advanced/fear_greed.py`	Live sentiment API & risk multiplier scaler.
Validator	`src/validator.py`	exchangeInfo caching & precision math.
Logger & Redactor	`src/logger.py`	Sanitizes API keys/secrets in bot.log.

3. Algorithmic Strategy Formulations

3.1 Quantity & Price Precision Quantization:

To prevent Binance API error code -1111 (Precision Over Maximum), all numerical parameters are passed through quantization routines based on exchange rules:

```
Quantized Qty = floor(Quantity / StepSize) * StepSize
```

3.2 TWAP Time Chunking Formula:

For total quantity Q over duration D minutes with N chunks, chunk size q and interval t are calculated as:

```
q = Quantize(Q / N, StepSize), t = (D * 60) / N seconds
```

3.3 Grid Level Matrix:

Given lower price *P_min*, upper price *P_max*, and grid count *G*, price step ΔP is:

```
ΔP = (P_max - P_min) / (G - 1)
```

4. Automated Testing & Verification Results

The automated test suite in tests/ was executed using PyTest. All 9 test modules passed successfully:

```
============================= test session starts ============================== platform
win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0 rootdir: D:\INS_PD collected 9 items
tests\test_advanced.py ... [ 33%] tests\test_client.py .. [ 55%] tests\test_validator.py
.... [100%] ============================== 9 passed in 0.34s
==============================
```

5. Audit Log Verification (bot.log)

Logs are continuously saved to bot.log. Below is a sanitized sample demonstrating credential redacting and structured logging:

```
2026-09-02 11:31:35 [INFO] binance_bot: Fetching Crypto Fear & Greed Index data...
2026-09-02 11:31:36 [INFO] binance_bot: Fear & Greed Index: 63 (Greed) 2026-09-02 11:08:45
[INFO] binance_bot: API Request: POST /fapi/v1/order | Params: {'symbol': 'BTCUSDT',
'side': 'BUY', 'type': 'MARKET', 'quantity': 0.01, 'timestamp': 1788307200, 'signature':
'[REDACTED_SIGNATURE]'} 2026-09-02 11:08:46 [INFO] binance_bot: API Response Success:
/fapi/v1/order
```